

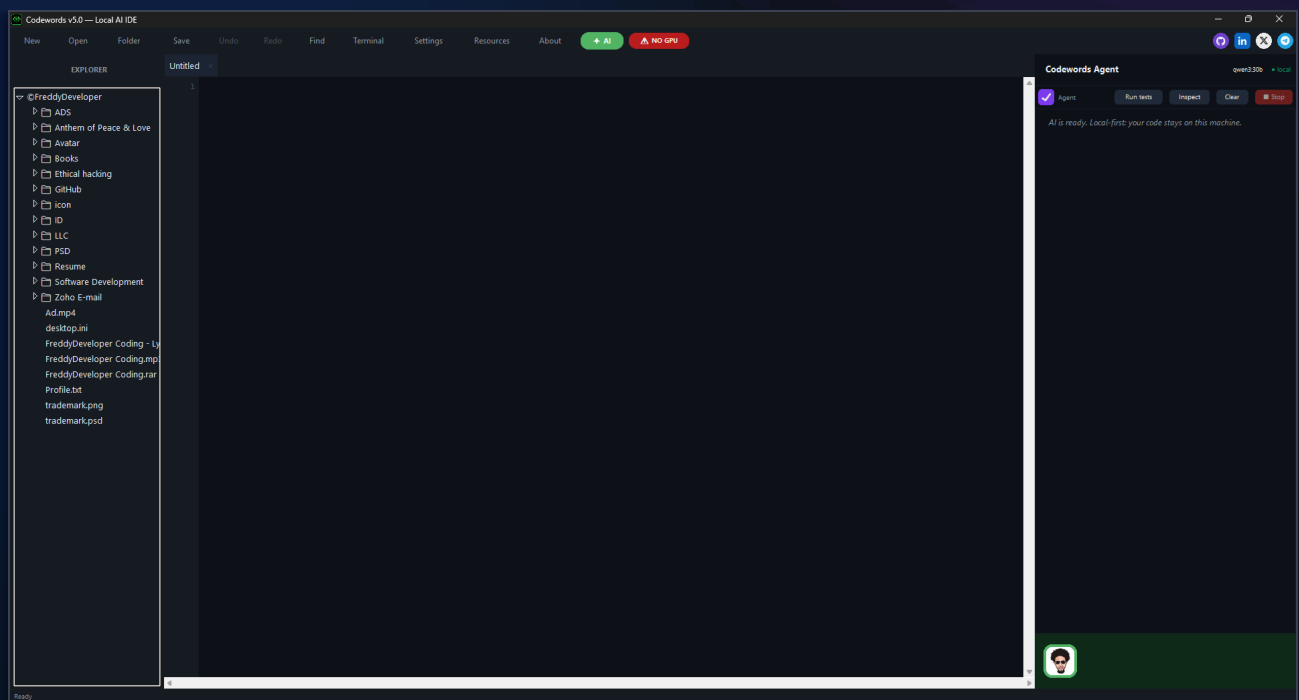
USER GUIDE · v5.0

Codewords

Your local AI coding partner.

An editor and an AI agent that lives on your own computer.

No cloud, no API keys, no monthly bill — your code never leaves your machine.



Windows app · Free download on GitHub · Powered by Ollama + Qwen3:30b

FreddyDeveloper

freddydeveloper.com · 2026

What's inside

This guide explains every part of Codewords in plain English. You don't need to know how AI works — if you can write a little code, you can use everything in here.

01	Welcome to Codewords What it is and why it's different
02	Getting started Install, first launch and the one-time setup
03	A tour of the window Every panel, button and bar
04	The code editor Tabs, highlighting, saving and zoom
05	Find & Replace Search, highlight and fix code in seconds
06	The built-in terminal Run commands without leaving the app
07	Meet your AI agent How it thinks and how to talk to it
08	The agent's toolbox Everything the agent is able to do
09	Planning, tasks & specialists How it handles big jobs
10	Memory, sessions & skills How it remembers your project
11	Safety & permissions You are always in control
12	Settings explained Every option, what it does
13	Keyboard shortcuts & files Speed tricks and the .codewords folder
14	Tips, FAQ & troubleshooting Get the best results

01

Welcome to Codewords

A code editor with a teammate built in

Codewords is a desktop code editor with an AI agent living inside it. You can write code yourself, exactly like in any editor, and whenever you want help you just ask the agent in plain words: “find the bug in the login”, “add a dark mode”, “write tests for this file”.

The big difference from tools like ChatGPT or online coding assistants: **the AI runs on your own computer**. It uses a free program called **Ollama** and an open model called **Qwen3:30b**. Nothing is uploaded anywhere, you don't need an account, and there's no API key or subscription.



Private by default

Your code stays on your disk. The AI talks to a server at 127.0.0.1 — that's your own computer, not the internet.



An agent, not just a chat

It doesn't only give advice. It can open files, search your project, edit code, run commands and tests — then check its own work.



You stay in control

Risky actions like deleting files or running dangerous commands always stop and ask you first. You can press Stop at any time.



Works offline

After the one-time model download, Codewords works with no internet at all — on a plane, in class, anywhere.

Who is this for?

Anyone who writes code: students, junior developers, freelancers and seniors who want a private assistant. If you understand what a file, a folder and a terminal command are, you have everything you need.

Good to know An AI agent is simply an AI that can **take actions** (use tools) instead of only answering. Codewords gives the model a set of safe tools and lets it work step by step, like a helpful coworker who reads your project before touching it.

02

Getting started

From zero to your first AI answer

What you need

Item	Details
Windows 10 / 11	Codewords is a ready-to-run Windows program (Codewords.exe). You don't need Python or anything else installed.
Internet (first launch)	Only needed once, to install Ollama and download the AI model. After that everything works offline.
Disk space	About 20 GB free. The Qwen3:30b model is a large download (roughly 19 GB), done only once.
Memory (RAM)	32 GB is recommended for a smooth experience. Less may work, but slower.
Graphics card (GPU)	Optional but strongly recommended. With a dedicated NVIDIA, AMD or Intel Arc GPU the AI answers much faster. Without one it still works, just slowly.

Download and install

1 Download from GitHub

Open the Codewords repository on github.com/FreddyDeveloper, go to **Releases** and download **Codewords.exe** from the latest version.

2 Put it in a folder

Move the file to a place you like, for example **Documents\Codewords**, then double-click it. There is no installer to run.

3 If Windows warns you

New apps may show "Windows protected your PC" (SmartScreen). Click **More info** → **Run anyway**. Only do this with the file from the official GitHub page.

4 Let the setup finish

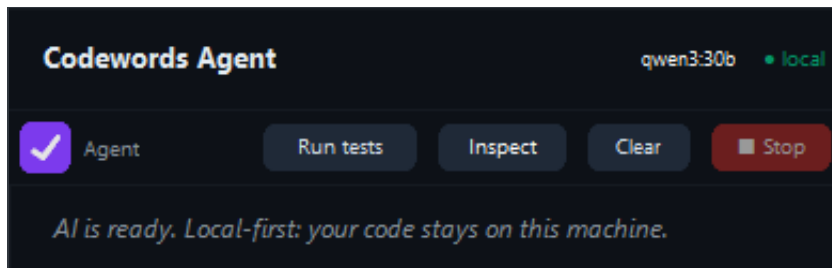
The first launch opens a setup window that installs **Ollama** (or updates it) and downloads **qwen3:30b**. Let it finish once — after that Codewords starts in seconds.

Updating To update, download the new **Codewords.exe** from Releases and replace the old one. Your settings, chats and notes are kept, because they live in `~/.codewords` and in each project's `.codewords` folder — not inside the `.exe`.

Your first 60 seconds

- Click **Folder** in the toolbar and choose your project. This becomes your **workspace** — the place the agent is allowed to work in.
- Double-click any file in the **Explorer** on the left to open it.
- Look at the AI panel on the right. When the header shows ● **local**, the AI is ready.
- Type “Explain what this project does” and press **Enter**. Watch the agent read your files and answer.

This is what the agent panel looks like when everything is ready:

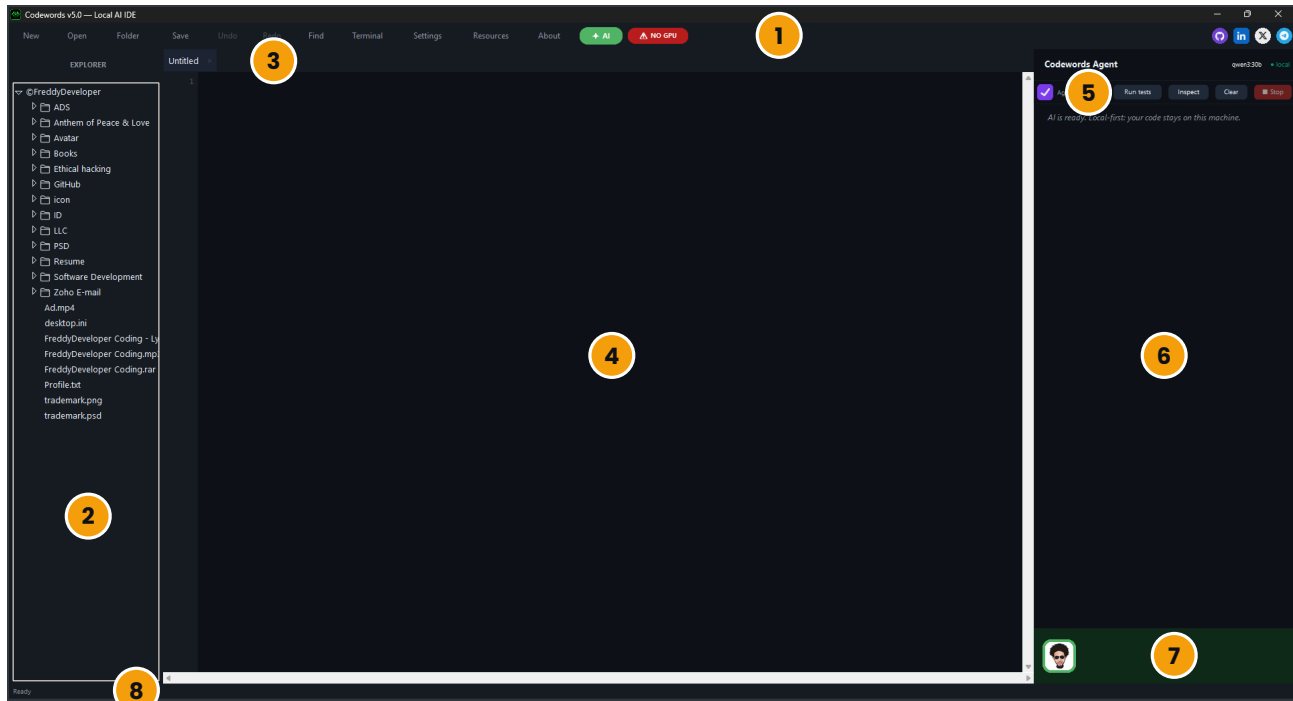


The green ● **local** at the top right means Ollama is running on your PC with the model loaded. The purple **Agent** checkbox is on, so the AI can use tools. If you see a red ● **Ollama offline** or an orange ● **model missing** instead, jump to Chapter 14.

03

A tour of the window

Eight areas, each with one job



Area	What it does
1 Toolbar	The main buttons: files, Undo / Redo (grey when there's nothing to undo), Find, Terminal, Settings and more. The green AI badge jumps to the chat box and turns amber (" Busy") while the agent works. The red NO GPU badge only appears if no graphics card was found.
2 Explorer	The file tree of your workspace (the folder you opened). Click to expand a folder, double-click a file to open it.
3 Tabs	One tab per open file. A dot means unsaved changes. Click to close.
4 Editor	Where you write code, with line numbers and color highlighting. The Find panel and the Terminal open just below it.
5 Agent controls	The Agent checkbox and the Run tests, Inspect, Clear and Stop buttons. Above them: the model name and the status (local).
6 Chat	The conversation with the agent: your messages, every tool it uses (in green) and its answers.
7 Message box	Type your request here and press Enter. Click your avatar to change the picture.
8 Status bar	Shows the language and name of the current file, plus short messages.

The toolbar, button by button

Button	What happens when you click it
New	Opens an empty Untitled tab. Shortcut Ctrl+N .
Open	Opens one file from anywhere on your PC. Shortcut Ctrl+O .
Folder	Chooses your workspace (project folder). The Explorer, the terminal and the AI all follow this folder, and each workspace keeps its own AI history.
Save	Saves the current tab. New files ask where to save. Ctrl+S saves, Ctrl+Shift+S is Save as.
Undo	Takes back your last change in the current file. Ctrl+Z . The button turns grey when there is nothing left to undo.
Redo	Brings back what you just undid. Ctrl+Y or Ctrl+Shift+Z .
Find	Opens the Find & Replace panel. Ctrl+F to find, Ctrl+H to replace.
Terminal	Shows or hides the built-in terminal. Ctrl+` .
Settings	AI model, agent limits, memory and permission options (Chapter 12).
Resources	A handy list of download links: languages, runtimes and developer tools.
About	Version, author and links.
♦ AI	Puts your cursor in the AI chat box. Ctrl+K .

No GPU badge If you see a red ⚠ **NO GPU** badge in the toolbar, Codewords didn't find a dedicated graphics card. Click it for details. Everything still works — answers will simply take longer.

Tip The social icons on the right of the toolbar (GitHub, LinkedIn, X, Telegram) open the author's pages in your browser.

04

The code editor

Comfortable, fast and familiar

The editor works like any editor you already know: type, select, copy, paste and undo. On top of that it adds a few helpers.

Syntax highlighting

Keywords, strings, comments, numbers and functions get their own colors so code is easier to read. Built-in color rules for Python, JavaScript, HTML, CSS, JSON and Rust.

Line numbers & current line

Numbers on the left help you follow AI reports like “error on line 42”. The line with your cursor is softly highlighted.

Unlimited undo & redo

Ctrl+Z undoes and **Ctrl+Y** redoes — as many steps as you want, also from the toolbar buttons. A paste or a big Replace All counts as one step. Each tab keeps its own history.

Zoom

Ctrl++ makes code bigger, **Ctrl+-** smaller. The size is remembered for next time.

Knows 25+ languages

The status bar recognizes Python, JS, TS, PHP, HTML, CSS, Rust, Go, Java, Kotlin, C, C++, C#, Ruby, SQL, Bash, PowerShell, YAML, TOML, Lua, Markdown and more.

Safe closing

If you close a tab with unsaved changes, Codewords asks: save, don't save, or cancel. You never lose work by accident.

Right-click menu Right-click inside the editor for **Undo**, **Redo**, **Cut**, **Copy**, **Paste**, **Delete**, **Select All**, **Find** and **Replace**. Options that can't be used right now (for example Copy with nothing selected) are greyed out.

Undo the AI's changes When the agent edits a file you have open, the editor updates it as **one undo step**. Press **Ctrl+Z** to see your previous version again, then **Save** if you want to keep it.

AI code opens in tabs When the agent answers with a code block, Codewords automatically opens that code in a **new tab** with highlighting and line numbers, so you can read it comfortably, copy it or save it.

05

Find & Replace

One of the strongest tools in Codewords

The Find panel is built for fixing code, not just locating a word. As you type, every match lights up in the editor, the current one is marked in orange and selected, and a list shows each match with its line number. Then you can replace one match, or all of them at once — just like Notepad, but much smarter.

Opening it

- Click **Find** in the toolbar, or press **Ctrl+F**. Press **Ctrl+H** to jump straight to the Replace box.
- If you had a word selected, it's already filled in for you.
- Press **Esc** or the **x** to close. Highlights disappear when the panel closes.

The four search options

Button	Name	What it does	Example
Aa	Match case	Upper and lower case must match exactly. Alt+C	User won't match user
ab	Whole word	Only matches complete words, not parts of longer words. Alt+W	id won't match width
.*	Regex	Use regular expressions for pattern searches. Alt+R	print\(.*\) finds every print call
Sel	In selection	Search only inside the code you selected — perfect for renaming a variable in one function only.	Select a function, then Find

Where to search

Scope	Searches in	Replace All does
This file	The tab you're looking at.	Changes this file in the editor. One Ctrl+Z undoes everything.
Open tabs	Every open tab, including unsaved ones.	Changes each tab in the editor (not saved yet, so you can review).
Workspace	Every text file in your project folder (skips binaries, big files and folders like node_modules and .git).	Asks for confirmation, writes files on disk and keeps a backup copy of every changed file.

Moving between matches

- **Enter** or **F3** goes to the next match; **Shift+Enter** or **Shift+F3** goes back. The ▲▼ buttons do the same.
- The counter (e.g. 3 of 12) tells you where you are. In **Workspace** mode it also says how many files matched.
- Click any line in the **results list** to open that file and jump right to the match, already selected.
- Keep editing while the panel is open — the highlights update by themselves.

Replacing

- **Replace** (or **Enter** in the Replace box) changes the orange match and moves to the next one, so you can check each change.
- **Replace All** (or **Ctrl+Enter**) changes every match in the chosen scope at once.
- With **Regex** on, you can reuse parts of the match: `\1` , `\2` ... or `\g<name>` .

Regex example

```
# Swap 'name = value' into 'value = name'   (Regex on)
Find:    (\w+) = (\d+)
Replace: \2 = \1

# Before:  timeout = 30           After:  30 = timeout
```

Undo & backups In the editor, one Replace All = one undo step. In **Workspace** mode, files that were not open are written to disk, and an exact copy of each original is saved in `.codewords/find-backups/<date-time>/` — so you can always get your old code back.

06

The built-in terminal

A real command line, right under your code

Press **Ctrl+`** (the key left of 1) or click Terminal. A panel opens under the editor with a live shell (PowerShell/cmd on Windows) that keeps running between commands — just like a normal terminal window.

Starts in your project

The terminal opens in your workspace folder, so commands like `python main.py` or `npm start` work right away.

Change folder

The folder button lets you pick any other folder. The current path is shown at the top.

Restart

Something stuck? Restart gives you a fresh shell in one click.

Ctrl+C to interrupt

Stops a running program (a server, a long script) without closing the terminal.

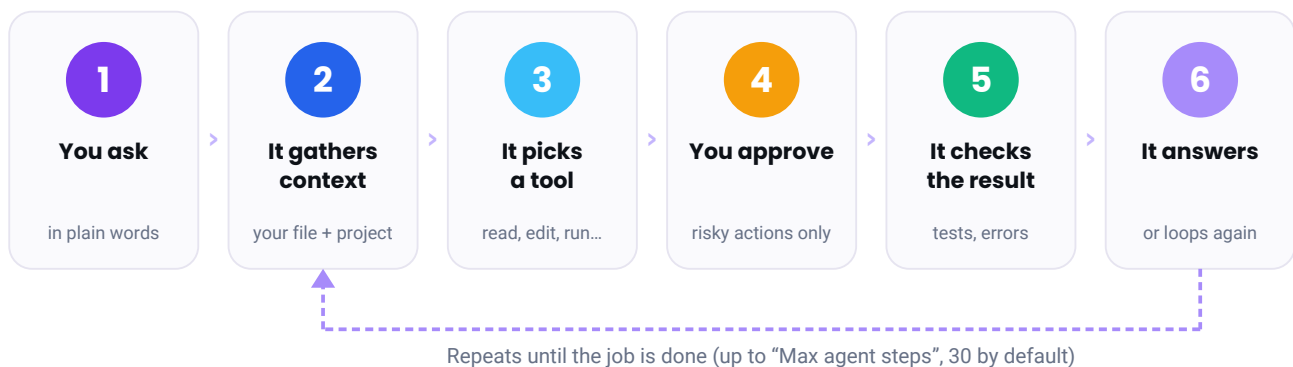
Terminal vs. the agent The terminal is **for you**. The agent runs its own commands separately (with its own safety rules) and shows their results in the AI panel.

07

Meet your AI agent

How it thinks, and how to talk to it

When you send a message, the agent doesn't answer from memory alone. It works in a loop: it looks at your project, uses a tool, looks at the result, and decides what to do next — until the job is finished.



What it already knows when you ask

Every message automatically includes useful **context**: the file you have open, the code you selected, your workspace path and a summary of the project. That's why you can say “fix this” without pasting code.

The AI panel controls

Control	What it does
Agent ☒	On (default): the AI can use tools — read, edit, run. Off: plain chat mode, it only talks.
Send	Sends your message. Enter sends, Shift+Enter adds a new line.
■ Stop	Cancels the current request immediately. Turns red while the agent is working.
Clear	Starts a fresh conversation. The old one stays saved on disk, but will not come back when you reopen the app.
Inspect	Pick a file and the agent writes a full error report: syntax errors, bugs, security problems and bad practices — each with line number, severity (critical / warning / info) and a fix.
Run tests	Finds your project's tests, runs them, fixes what fails and runs them again.
Avatar	Click your picture to change it. Just for fun — it's your chat profile photo.

Reading the status

You see	Meaning
● local	Ollama is running on your PC and the model is ready.
● model missing	Ollama works, but the model in Settings isn't downloaded yet.
● Thinking...	The agent is working. An amber bar moves at the top of the panel and a timer counts seconds.
tool lines	Green lines like <code>read_file src/app.py</code> show each action as it happens — nothing is hidden.
● Responding...	The answer is being written in the chat, word by word.
● Ollama offline	Ollama isn't reachable. See Troubleshooting (Chapter 14).

Shortcut commands

Type	Same as asking
<code>/test</code>	Run the most appropriate tests for this project, fix failures, and verify again.
<code>/fix</code>	Inspect the current file/project for errors, fix them, and verify the result.

How to ask for great results

✗ Vague

"it doesn't work"

✓ Clear

"Clicking Save in settings.py throws KeyError: 'theme'. Find why and fix it."

✗ Too big at once

"build me a whole online store"

✓ Step by step

"First plan the folder structure for a small store with products and cart. Don't write code yet."

- Say what, where and how you'll know it works: the file, the behavior you expect, the error message.
- Select code first if your question is about a specific part — the selection is sent with your message.
- Ask it to explain: "Explain this function like I'm a junior developer" works great for learning.

08

The agent's toolbox

Everything the agent can do for you

These are the “hands” of the agent. You never call them yourself — the agent chooses the right one. Knowing them helps you understand the green tool lines in the chat and ask for the right things.

Explore the project

`list_files` see folders and files
`find_files` find files by pattern, e.g. `**/*.py`
`read_file` read a file (or some lines)
`read_many_files` read several files together
`search_workspace` search text in all files
`inspect_project` detect languages, frameworks, tests
`project_context` read README and project notes

Edit code

`write_file` create or overwrite a file
`edit_file` change one exact piece of code
`replace_all` replace every occurrence in a file
`apply_patch` apply a multi-part change (diff)
`create_directory` make a folder
`move_file` / `copy_file` rename, move or copy
`delete_file` delete (always asks you)

Run & check

`run_command` run a shell command
`run_tests` detect and run your tests
`get_diagnostics` check syntax / types / build
`format_code` run your code formatter

Git (version control)

`git_status` what changed
`git_diff` exact line changes
`git_log` / `git_show` commit history
`git_blame` who changed each line

Understand code

`search_symbols` find where a function/class is defined
`find_references` find everywhere it's used
`build_context` gather the most relevant code

Organize & remember

`create_task` / `update_task` track multi-step work
`enter_plan_mode` switch to read-only planning
`write_memory` / `read_memory` remember project facts
`delegate_specialist` ask an expert helper

PDFs too If you ask the agent to create a file ending in `.pdf`, it builds a real PDF document (titles, subtitles and bullet lists included) instead of a text file.

Smart tool loading Small local models work better with fewer choices. In **auto** mode, Codewords only adds the advanced tools (references, skills, context builder) when your request is long or complex. You can change this in Settings.

09

Planning, tasks & specialists

How the agent handles big jobs

Large requests are easier when they're broken into pieces. Codewords gives the agent three tools for that.

1 Plan mode — think before touching anything

In plan mode the agent is **read-only**: it can explore and read, but it cannot write files or run commands that change your project. It studies the code and gives you a plan. When the plan is ready it leaves plan mode and starts working. Try: "Plan how to add user login. Don't change anything yet."

2 Tasks — a to-do list that survives restarts

For multi-step work the agent creates tasks like "Create database model" or "Add tests". Each task has a status: **pending**, **in_progress**, **completed**, **blocked** or **cancelled**. Tasks can depend on each other — finishing one automatically unblocks the next. They're saved in `.codewords/tasks.json`, so work can continue tomorrow.

3 Specialists — expert helpers

The main agent can ask a focused helper to analyze your code. Specialists are **read-only** (they can look, never change) and each has one job:

Specialist	Its job
Explorer	Maps the architecture: important files, entry points, dependencies and conventions.
Architect	Designs an implementation plan that fits your existing code.
Debugger	Follows the evidence to find the root cause of a bug and likely fixes.
Reviewer	Reviews code for correctness, edge cases, security and maintainability.
Tester	Decides the best tests and diagnostics and the likely failure cases.

10

Memory, sessions & skills

How the agent remembers your project

Sessions — your conversations

Each workspace keeps its own chat history in `.codewords/sessions/` (the last 20 conversations). When you open Codewords again, your last conversation comes back so you can continue where you left off. Press **Clear** to start fresh — the next time you open the app it starts clean. Very long chats are automatically shortened so the local model doesn't get overloaded.

Memory — things worth remembering

The agent can save **durable notes** about your project: architecture, conventions, gotchas (“this project uses tabs, not spaces”). It reads them in future sessions. It's told never to store secrets. Memory lives in one of three places:

Scope	Saved in	Use it for
Project	<code>.codewords/agent-memory/</code>	Notes about this project that you might share with your team (default).
Local	<code>.codewords/agent-memory-local/</code>	Notes about this project that are only for your computer.
User	<code>~/.codewords/agent-memory/</code>	Notes about you that apply to every project.

Symbol index — a map of your code

When you open a workspace, Codewords quickly lists every function, class and type in your project and saves it to `.codewords/index.json`. This lets the agent jump straight to where something is defined and used instead of reading every file.

Skills — teach the agent your rules

Skills are short instruction sheets the agent loads only when relevant. Built-in skills cover Python, JavaScript, TypeScript, Rust, PHP, web, Godot, Git, debugging, testing and security. Opening a `.py` file automatically loads the Python skill, for example.

You can write your own! Create a Markdown file and the agent can use it:

`my-style.md`

```
# Project skill (only this project)  .codewords/skills/my-style.md
# Personal skill (all projects)      ~/.codewords/skills/my-style.md
```

```
Always use snake_case for variables.
Every new function needs a docstring and a unit test.
Never use print() for logs; use the logging module.
```

11

Safety & permissions

The agent is powerful — and you have the final word

Before running any command, Codewords classifies it into one of four risk levels. The level decides whether it runs automatically or asks you first.

READ

Runs automatically

ls, git status, cat

WRITE

Auto if you allow it

mkdir, git commit, mv

NETWORK

Asks by default

pip install, git pull

DANGEROUS

ALWAYS asks you

rm -rf, DROP TABLE

See the change before it happens

When approval is on, file edits show you a **diff** — red lines removed, green lines added — before anything is saved.

Stays inside your workspace

The agent can't read or write files outside the project folder you chose.

Deleting always asks

Deleting files and dangerous commands (`rm -rf`, `git reset --hard`, `DROP TABLE` ...) always need your “Yes”.

Stop button

Changed your mind? Press **■** Stop and the agent halts immediately.

Default behavior Out of the box, **safe commands run automatically** and **file edits are applied automatically** so the agent can work fast. If you prefer to approve every edit, turn off “Automatically apply file edits” in Settings. Network commands (installs, downloads, git pull/push) ask by default.

Tip for beginners Use Git in your project. Then, whatever the agent changes, you can always see it with `git diff` and undo it.

12

Settings explained

Every option, in plain words

Setting	Default	What it means
Ollama URL	127.0.0.1:11434	Where the AI server lives. Leave it unless you run Ollama elsewhere.
Model	qwen3:30b	Which AI model to use. Any model you have in Ollama works; bigger = smarter but slower.
Context window (files)	12	How many project files the agent considers when gathering context.
Max agent steps	30	The most tool actions the agent may take for one message. Raise it for big jobs.
Max specialist steps	8	The same limit, for each specialist helper.
Context budget	50,000 chars	How much code the context builder may collect at once.
Chat history kept	120,000 chars	How much past conversation is remembered before older parts are summarized.
Model context size	65,536 tokens	How much text the model can “see” at once. Higher uses more memory.
Advanced tools	auto	auto : only for complex requests. on : always. off : never.
Memory scope	project	Where new memories are stored: project, local or user (Chapter 10).
Enable skills	on	Let the agent load skills.
Index symbols automatically	on	Build the code map when a workspace opens.
Save and restore chat sessions	on	Keep conversations per workspace.
Auto-run non-destructive commands	on	Read and write commands run without asking.
Auto-run network commands	off	Installs, downloads and git pull/push ask first.
Automatically apply file edits	on	Turn off to review a diff and approve every edit.

Settings are saved in `~/.codewords/config.json` (your user folder). Dangerous commands always ask, whatever you choose here.

13

Keyboard shortcuts & files

Speed tricks and what Codewords saves

Ctrl+N	New file	Ctrl+O	Open file
Ctrl+S	Save	Ctrl+Shift+S	Save as
Ctrl+F	Find	Ctrl+H	Find & Replace
F3 / Shift+F3	Next / previous match	Esc	Close Find panel
Ctrl+K	Jump to AI chat	Enter	Send message
Shift+Enter	New line in chat	Ctrl+J	Show / hide AI panel
Ctrl+B	Show / hide Explorer	Ctrl+`	Show / hide Terminal
Ctrl+ + / Ctrl+ -	Zoom in / out	Ctrl+Z / Ctrl+Y	Undo / redo
Ctrl+Shift+Z	Redo (alternative)	Ctrl+A	Select all

Inside the Find panel

Enter next · **Shift+Enter** previous · **Ctrl+Enter** (in Replace) Replace All · **Alt+C** match case · **Alt+W** whole word · **Alt+R** regex

The .codewords folder

Codewords keeps its project data in a hidden **.codewords** folder inside your workspace. It's safe to delete — you'd only lose history, notes and backups.

Folder layout

```

your-project/
├── .codewords/
│   ├── sessions/           # your AI conversations (last 20)
│   ├── tasks.json          # the agent's to-do list
│   ├── index.json          # map of functions and classes
│   ├── agent-memory/       # project notes the agent saved
│   ├── agent-memory-local/ # private notes for this PC
│   ├── skills/             # your custom skills
│   └── find-backups/       # copies made by Workspace Replace All
└── ~/.codewords/          # in your user folder
    ├── config.json         # your settings
    ├── skills/
    └── agent-memory/
  
```

Using Git? Add **.codewords/** to your **.gitignore** if you don't want to share sessions and notes. Codewords already hides it from Git locally for you.

14

Tips, FAQ & troubleshooting

Answers to the questions everyone asks

Q. The AI panel says ● Ollama offline.

Ollama isn't running. Open the Ollama app (or run `ollama serve` in a terminal), then check Settings: the URL should be `http://127.0.0.1:11434` and the model name must match one you have (`ollama list` shows them).

Q. Answers are very slow.

That usually means no GPU, or not enough RAM for a 30B model. Close heavy apps, or try a smaller model in Settings (for example a 7B or 14B coder model you've pulled with Ollama).

Q. The agent stopped before finishing.

It reached **Max agent steps**. Just reply "continue", or raise the limit in Settings.

Q. I pressed Clear but want the old chat back.

It's still saved in `.codewords/sessions/` as a JSON file — nothing is deleted.

Q. Can I undo what the agent changed?

Yes. In an open tab, `Ctrl+Z` undoes the agent's whole edit in one step (then Save). Otherwise, use Git (`git diff` / `git restore`). For Workspace Replace All, use the copies in `.codewords/find-backups/`.

Q. Does anything go to the internet?

The AI runs locally. Internet is only used for the one-time setup (Ollama and the model) and if you approve network commands like `pip install`.

Q. Can I use another model?

Yes. Pull it with `ollama pull <name>` and type the name in Settings → Model. Models that support tool calling work best with Agent mode.

Q. Does it work on Mac or Linux?

No. Codewords is a Windows `.exe`, and its automatic setup is made for Windows.

Q. Windows says the app is unsafe.

That's SmartScreen, which warns about new programs that aren't signed with a paid certificate. If you downloaded Codewords.exe from the official GitHub Releases page, click More info → Run anyway.

Five habits of happy Codewords users

- Open the project folder, not single files — the agent works best when it can see the whole project.
- Commit with Git before big AI changes, so you can compare and undo.
- Ask for a plan first on big tasks, then say "go ahead".
- Use **Inspect** on any file before shipping it — it's a free code review.
- Use **Find & Replace** for quick, precise renames; use the agent when the change needs understanding.

Happy coding.

Write it. Ask it. Ship it — all on your own machine.

Codewords v5.0 · FreddyDeveloper · freddydeveloper.com
github.com/FreddyDeveloper · t.me/freddydeveloper · x.com/freddydeveloper